

AUTOM-06: Dynatrace SDKs

Series: AUTOM — Dynatrace Automation | **Notebook:** 6 of 9 | **Created:** January 2026 | **Last Updated:** 10/02/2026

Dynatrace publishes official TypeScript SDK clients (`@dynatrace-sdk/*`) for programmatic access to the platform, auto-generated from OpenAPI specifications. There is no official Python SDK package — Python automation calls the REST APIs directly.

Table of Contents

- 1. [Introduction](#)
- 2. [TypeScript SDK](#)
- 3. [Python: Calling the REST APIs](#)
- 4. [Common Patterns](#)
- 5. [MCP Server Integration](#)
- 6. [Next Steps](#)

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Node.js 18+	For TypeScript SDK
Python 3.9+ with <code>requests</code>	For the Python REST examples
API Token	Token with required scopes
Development Environment	VS Code or similar IDE

Learning Objectives

By the end of this notebook, you will:

- Understand the Dynatrace SDK architecture
 - Know how to use the TypeScript SDK clients and call the REST APIs from Python
 - Be able to query data and manage configurations programmatically
 - Build custom automation applications
-

1. Introduction

Available SDKs

SDK	Language	Use Case
@dynatrace-sdk/*	TypeScript/JavaScript	Dynatrace apps, app functions, workflow JavaScript tasks
(none)	Python	No official Python SDK is published (<code>dt-sdk</code> is not on PyPI). Call the REST APIs with <code>requests</code> or a clearly labelled community library

SDK vs Direct API

Aspect	SDK	Direct API
Type safety	Yes	No
Auto-completion	Yes	No
Authentication	Built-in	Manual
Pagination	Cursor exposed (<code>nextPageKey</code>); you loop	Manual
Error handling	Structured	Raw HTTP

Client Libraries

Client (package)	Purpose
<code>queryExecutionClient</code> (<code>client-query</code>)	Execute DQL queries (<code>queryExecute</code> , then <code>queryPoll</code>)
<code>settingsObjectsClient</code> (<code>client-classic-environment-v2</code>)	Manage Settings 2.0 objects
<code>monitoredEntitiesClient</code> (<code>client-classic-environment-v2</code>)	Query entity topology
<code>metricsClient</code> (<code>client-classic-environment-v2</code>)	Query and ingest metrics
<code>eventsClient</code> (<code>client-classic-environment-v2</code>)	Query and create events
<code>logsClient</code> (<code>client-classic-environment-v2</code>)	Query and store logs

Names are the exports of `@dynatrace-sdk/client-query` 1.27.0 and `@dynatrace-sdk/client-classic-environment-v2` 9.1.0 ([client-query reference \(Dynatrace Developer\)](#), [client-classic-environment-v2 reference \(Dynatrace Developer\)](#)); a list call returns `nextPageKey` for the caller to follow.

2. TypeScript SDK

Installation

```
npm install @dynatrace-sdk/client-query
npm install @dynatrace-sdk/client-classic-environment-v2
```

(`@dynatrace-sdk/client-core` and `@dynatrace-sdk/client-settings-v2` are not published on npm; Settings 2.0 objects are reached through `client-classic-environment-v2` .)

Configuration

The `@dynatrace-sdk/*` clients are built for code that runs on the Dynatrace platform — apps, app functions and workflow JavaScript tasks — where the platform runtime supplies authentication. There is no `setEnvConfig` step to call. For scripts that run outside the platform (CI jobs, local tooling), call the REST APIs directly as in §3.

Query Example

`queryExecute` starts the query and returns its result only if it finishes within `requestTimeoutMilliseconds`; otherwise it returns `state: 'RUNNING'` and a `requestToken` to poll with `queryPoll`. Put the timeframe in the DQL (`from:`) — `defaultTimeframeStart` / `defaultTimeframeEnd` take ISO-8601 timestamps (*"The query timeframe 'start' timestamp in ISO-8601 or RFC3339 format"*), not relative strings such as `now-24h`. ([client-query reference \(Dynatrace Developer\)](#))

```

import { queryExecutionClient } from '@dynatrace-sdk/client-query';

// Start a DQL query, then poll until it finishes
async function runQuery(query: string) {
  const start = await queryExecutionClient.queryExecute({
    body: { query, requestTimeoutMilliseconds: 30000 },
  });

  let { state, result } = start;
  while (state === 'RUNNING' || state === 'NOT_STARTED') {
    ({ state, result } = await queryExecutionClient.queryPoll({
      requestToken: start.requestToken!,
      requestTimeoutMilliseconds: 30000,
    }));
  }
  if (state !== 'SUCCEEDED') {
    throw new Error(`Query ended in state ${state}`);
  }
  return result?.records ?? [];
}

async function queryHosts() {
  const records = await runQuery(`
    fetch dt.entity.host
    | fieldsAdd name = entity.name, osType
    | sort name asc
    | limit 10
  `);

  console.log('Hosts:', records);
  return records;
}

```

Settings Management

Schema-agnostic helpers: list every object of a schema (following `nextPageKey`), and create an object after a `validateOnly` dry run. Pass a schema that exists in your environment — management zones (`builtin:management-zones`) are **Blocked** at upgrade and not available in Latest Dynatrace (AUTOM-02 § 2), so prefer a current schema such as `builtin:ownership.teams`. Fetch the schema first (`GET /api/v2/settings/schemas/<schemaId>`) to build a valid `value`.

```

import { settingsObjectsClient } from '@dynatrace-sdk/client-classic-environment-v2';

// List all objects of one schema, following nextPageKey
async function listSettings(schemaId: string) {
  const items = [];
  let nextPageKey: string | undefined;
  do {
    const page = nextPageKey
      ? await settingsObjectsClient.getSettingsObjects({ nextPageKey })
      : await settingsObjectsClient.getSettingsObjects({ schemaIds: schemaId,
        pageSize: 500 });
    items.push(...(page.items ?? []));
    nextPageKey = page.nextPageKey;
  } while (nextPageKey);
  return items;
}

// Create one object – validate first, then write
async function createSetting(schemaId: string, scope: string, value:
Record<string, unknown> ) {
  const body = [{ schemaId, scope, value }];
  await settingsObjectsClient.postSettingsObjects({ body, validateOnly: true
});
  return settingsObjectsClient.postSettingsObjects({ body });
}

```

Entity Queries

```

import { monitoredEntitiesClient } from '@dynatrace-sdk/client-classic-environment-v2';

async function getServices() {
  const response = await monitoredEntitiesClient.getEntities({
    entitySelector: 'type(SERVICE)',
    fields: '+properties,+tags',
    pageSize: 100
  });

  return response.entities;
}

```

3. Python: Calling the REST APIs

Dynatrace does not publish a Python SDK: `pip install dt-sdk` fails because no such package exists on PyPI. Python automation calls the documented REST APIs directly. The examples below use the Settings 2.0 and Entities endpoints of the Environment API v2 with an access token (`Api-Token` header).

Setup

```
pip install requests
```

```
import os
import requests

BASE = os.environ['DT_TENANT_URL'].rstrip('/')          #
https://{tenant}.live.dynatrace.com
session = requests.Session()
session.headers['Authorization'] = f"Api-Token {os.environ['DT_API_TOKEN']}"
```

Settings Management

```
def list_settings_objects(schema_id: str) -> list[dict]:
    """All objects of one Settings 2.0 schema, following nextPageKey."""
    items: list[dict] = []
    params = {'schemaIds': schema_id, 'pageSize': 100}
    while True:
        r = session.get(f'{BASE}/api/v2/settings/objects', params=params,
            timeout=30)
        r.raise_for_status()
        body = r.json()
        items.extend(body.get('items', []))
        if not body.get('nextPageKey'):
            return items
        # With nextPageKey set, the API requires all other query parameters to
        # be omitted
        params = {'nextPageKey': body['nextPageKey']}

def create_settings_object(schema_id: str, value: dict, scope: str =
    'environment') -> list[dict]:
    r = session.post(f'{BASE}/api/v2/settings/objects',
        json=[{'schemaId': schema_id, 'scope': scope, 'value':
            value}], timeout=30)
    r.raise_for_status()
    return r.json()
```

Pick a schema that survives the upgrade to Latest Dynatrace for new automation (e.g. `builtin:davis.anomaly-detectors`); management zones and alerting profiles are on Dynatrace's removed-schemas list (AUTOM-02).

Pagination Handling

```
def get_all_hosts() -> list[dict]:
    """Iterate through all pages of hosts."""
    hosts: list[dict] = []
    params = {'entitySelector': 'type(HOST)', 'pageSize': 500}
    while True:
        r = session.get(f'{BASE}/api/v2/entities', params=params, timeout=30)
        r.raise_for_status()
        body = r.json()
        hosts.extend(body.get('entities', []))
        if not body.get('nextPageKey'):
            return hosts
        params = {'nextPageKey': body['nextPageKey']}
```

4. Common Patterns

Bulk Configuration Export

```
// TypeScript: collect all settings objects for backup (app function or
workflow task)
import { settingsObjectsClient, settingsSchemasClient } from '@dynatrace-
sdk/client-classic-environment-v2';
import type { SettingsObject } from '@dynatrace-sdk/client-classic-
environment-v2';

async function exportAllSettings() {
  // Get all available schemas
  const schemas = await settingsSchemasClient.getAvailableSchemaDefinitions();

  const exportData: Record<string, SettingsObject[]> = {};

  for (const { schemaId } of schemas.items) {
    if (!schemaId) continue;
    let page = await settingsObjectsClient.getSettingsObjects({ schemaIds:
schemaId, pageSize: 500 });
    const objects = [...page.items];
    // Follow the cursor; with nextPageKey set, omit the other query
parameters
    while (page.nextPageKey) {
      page = await settingsObjectsClient.getSettingsObjects({ nextPageKey:
page.nextPageKey });
      objects.push(...page.items);
    }
    if (objects.length > 0) {
      exportData[schemaId] = objects;
    }
  }

  // The platform runtime has no usable file system: return the data
  // (or store it in a document) instead of writing a file
  return exportData;
}
```

The SDK clients run on the Dynatrace platform, where `fs` is a stub: "You can import the following modules to ensure compatibility with specific third-party packages, but all exposed functions throw errors when called." Return the export (or store it in a document) rather than writing a file. To write a backup file from a CI job, use the Python REST pattern

in §3, which follows `nextPageKey` the same way. ([JavaScript runtime \(Dynatrace Developer\)](#))

Error Handling

```
async function safeQuery(query: string) {
  try {
    const records = await runQuery(query); // runQuery from §2
    return { success: true, data: records };
  } catch (error) {
    // Log and surface the failure; re-throw anything you cannot handle
    console.error('Query failed:', error instanceof Error ? error.message :
error);
    return { success: false, error: String(error) };
  }
}
```

Rate Limiting

```
import time
from functools import wraps

def rate_limited(max_per_second: float):
    """Decorator to rate limit API calls."""
    min_interval = 1.0 / max_per_second
    last_called = [0.0]

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_called[0]
            if elapsed < min_interval:
                time.sleep(min_interval - elapsed)
            result = func(*args, **kwargs)
            last_called[0] = time.time()
            return result
        return wrapper
    return decorator

@rate_limited(10) # Max 10 requests per second
def api_call(entity_id: str) -> dict:
    r = session.get(f'{BASE}/api/v2/entities/{entity_id}', timeout=30)
    r.raise_for_status()
    return r.json()
```

Async Operations

```
// TypeScript: concurrent queries with a concurrency cap
import pLimit from 'p-limit';

const limit = pLimit(5); // Max 5 queries in flight

async function queryMultipleHosts(hostIds: string[]) {
  const queries = hostIds.map(hostId =>
    limit(() => runQuery(`
      fetch dt.entity.host
      | filter id == "${hostId}"
    `)) // runQuery from §2
  );

  return Promise.all(queries);
}
```

Reporting Script

```
import pandas as pd

def generate_host_report() -> pd.DataFrame:
    """Generate a host inventory report from the Entities API (uses
    session/BASE from §3)."""
    rows = []
    params = {'entitySelector': 'type(HOST)', 'fields': '+properties',
'pageSize': 500}
    while True:
        r = session.get(f'{BASE}/api/v2/entities', params=params, timeout=30)
        r.raise_for_status()
        body = r.json()
        for e in body.get('entities', []):
            props = e.get('properties', {})
            rows.append({
                'name': e.get('displayName'),
                'osType': props.get('osType'),
                'cpuCores': props.get('cpuCores'),
                'physicalMemory': props.get('physicalMemory'),
            })
        if not body.get('nextPageKey'):
            break
        params = {'nextPageKey': body['nextPageKey']}

    df = pd.DataFrame(rows).sort_values('name')
    df.to_csv('host_inventory.csv', index=False)
    return df
```

Best Practices

Practice	Description
Environment variables	Never hardcode credentials
Type safety	Use the SDK's TypeScript types, or Python type hints on your REST wrappers
Error handling	Catch and handle API errors

Practice	Description
Pagination	Always handle paginated responses
Rate limiting	Respect API limits
Logging	Add structured logging

5. MCP Server Integration

Dynatrace MCP Server

The [Dynatrace MCP server \(DT docs\)](#) is hosted in your Dynatrace environment and gives AI assistants (MCP clients) a set of Dynatrace tools through the Model Context Protocol. There is nothing to install or run locally.

Connection

Point the client at `https://{environment-name}.apps.dynatrace.com/platform-reserved/mcp-gateway/v0.1/servers/dynatrace-mcp/mcp`. *"Every request to the MCP server needs a bearer token in the authorization header."* The docs recommend a platform token, *"because a token generated from an OAuth client is short-lived."*

```
{
  "servers": {
    "dynatrace-mcp": {
      "url": "https://{environment-name}.apps.dynatrace.com/platform-reserved/mcp-gateway/v0.1/servers/dynatrace-mcp/mcp",
      "headers": {
        "Authorization": "Bearer <platform-token>"
      }
    }
  }
}
```

This is the VS Code form shown on the docs page; other clients wrap the same `url` and `headers` in their own format (Claude Code, for example, uses an `mcpServers` entry with `"type": "http"`). Both the user and the token need `mcp-gateway:servers:invoke` and

`mcp-gateway:servers:read` , plus the permissions of the tools you call. Where the client supports it, read the token from a secret store rather than writing it into the file.

The local `@dynatrace-oss/dynatrace-mcp-server` npm package is deprecated. Its README states: *"This repository is deprecated. Version 2.1.2 was the final release — no further updates will be made."* The GitHub repository is archived. Move `npx -` based configurations to the hosted server above. ([dynatrace-mcp README](#) ([Dynatrace GitHub](#)))

MCP Server Capabilities

The use cases the docs page lists for the hosted server:

Capability	Description
DQL generation and explanation	Generate a DQL query, or explain one, with generative AI
DQL execution	Run a generated DQL query
Product questions	Answer product-related questions with generative AI
Problems and vulnerabilities	Investigate detected problems and vulnerabilities
Kubernetes events	Analyze Kubernetes events
Forecasting	Forecast and analyze timeseries data
Documents	Find documents and troubleshooting guides
Entities	Resolve entity names and IDs

When to Use MCP vs SDKs

Scenario	Tool Choice
AI-assisted development	MCP Server
Custom application	TypeScript SDK (platform apps) or REST API
Automated pipeline	REST API or Monaco/Terraform in CI/CD
Interactive triage	MCP Server with Claude or Copilot

Scenario	Tool Choice
Batch processing	SDK

6. Next Steps

When to Use SDKs

Scenario	Tool Choice
Custom reporting tool	SDK
One-off query	Direct API or Notebooks
Config deployment	Monaco or Terraform
Integration app	SDK
Complex logic	TypeScript SDK, or Python against the REST API
AI-assisted access	MCP Server

Continue the Series

Next Notebook	Focus
AUTOM-07: CI/CD Integration	GitOps patterns and pipelines

Additional Resources

- [Dynatrace Developer Portal](#)
- [TypeScript SDK Documentation](#)
- [Settings 2.0 API \(DT docs\)](#)
- [API Reference](#)
- [MCP Server Documentation](#)

Summary

In this notebook, you learned:

- How to use the TypeScript SDK clients, and how to call the REST APIs from Python
- Common patterns for queries, settings, and entities
- Building custom CLI tools and reporting scripts
- Dynatrace MCP Server for AI-assisted programmatic access
- Best practices for SDK usage

Key Takeaway: SDKs provide the most flexibility for custom applications. Use them when you need programmatic access, complex logic, or integration with other systems. For AI-assisted workflows, consider the MCP Server. For simple config management, Monaco or Terraform are often simpler.

Continue to AUTOM-07: CI/CD Integration to learn GitOps patterns.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.